Chapter 10: Data Subsampling

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Section 10.1: Support Points-based Subsampling **1**

3 Section 10.2: Data Splitting **2**

4 Section 10.3: Data Twinning **3**
4.1 Basic Twinning Example 3
4.2 Comparison of Subsampling Methods 4

5 Section 10.4: Supervised Compression **7**

6 Summary **8**

1 Introduction

This chapter covers data subsampling methods including support points-based subsampling, data splitting, and data twinning.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering*. CRC Press.

```
library(support)
library(SPlit)
library(MASS)
library(twinning)
library(BalancedSampling)
```

2 Section 10.1: Support Points-based Subsampling

```
set.seed(123)
N <- 1000
p <- 2
rho <- 0.5
Sigma <- diag(p)
Sigma <- rho^abs(row(Sigma) - col(Sigma))
D <- mvrnorm(n = N, mu = rep(0, p), Sigma = Sigma)

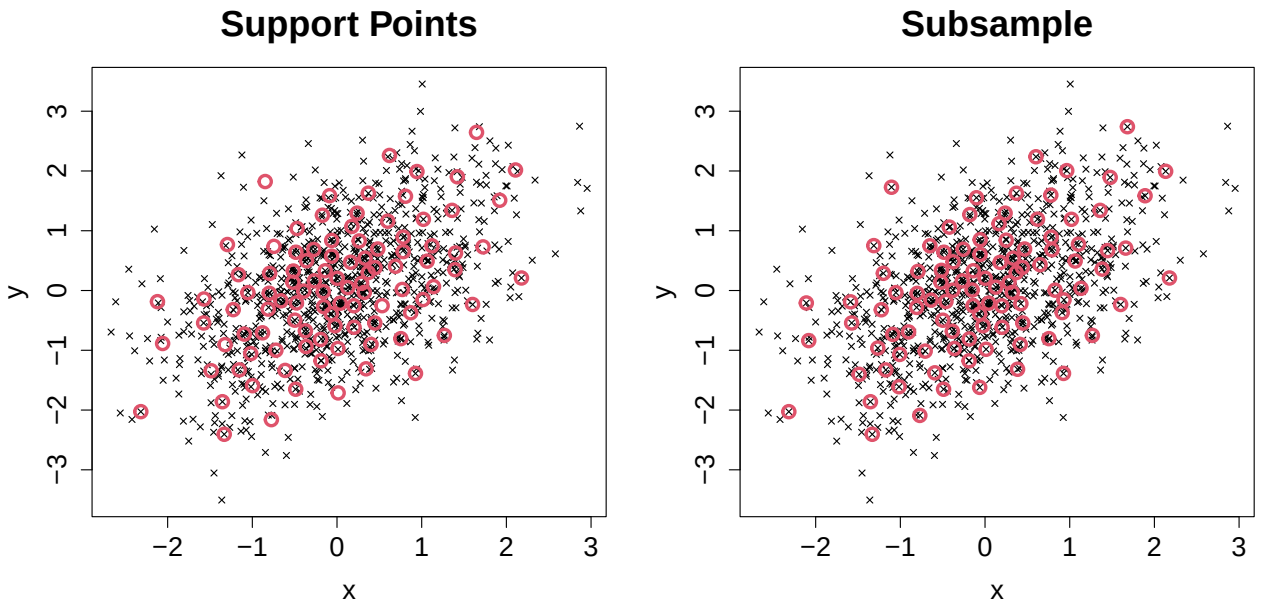
n <- 100
S <- sp(n, p, dist.samp = D)$sp
```

```
## Optimizing ...
## [>                                                                    ] 0 %[>
## Numerical instablities encountered... resetting optimization
## [=====================================================================>    ] 93 %[=======================
## Maximum iterations reached: done!
```

```r
par(mfrow = c(1, 2))

plot(D, pch = 4, cex = 0.75, lwd = 1, main = "Support Points",
     cex.main = 2, xlab = "x", ylab = "y", cex.lab = 1.5, cex.axis = 1.5)
points(S, pch = 1, col = 2, lwd = 3, cex = 1.5)

ind <- subsample(D, S)
plot(D, pch = 4, cex = 0.75, lwd = 1, main = "Subsample",
     cex.main = 2, xlab = "x", ylab = "y", cex.lab = 1.5, cex.axis = 1.5)
points(D[ind, ], pch = 1, col = 2, lwd = 3, cex = 1.5)
```




```r
par(mfrow = c(1, 1))
```

3 Section 10.2: Data Splitting

```r
set.seed(102)
N <- 100
x1 <- scale(rnorm(N))
x2 <- scale(x1^2 + rnorm(N))
D <- matrix(cbind(x1, x2), ncol = 2)
Ntest <- 20

y <- rep(2, N)
y[rbinom(N, 1, pnorm(-(6*x1 + x2 + 5))) == 1] <- 1
y[rbinom(N, 1, pnorm(-(-6*x1 + x2 + 5))) == 1] <- 3

Dy <- data.frame(x1 = D[, 1], x2 = D[, 2], y = as.factor(y))
ind.opt <- SPlit(Dy, splitRatio = 0.2)
```

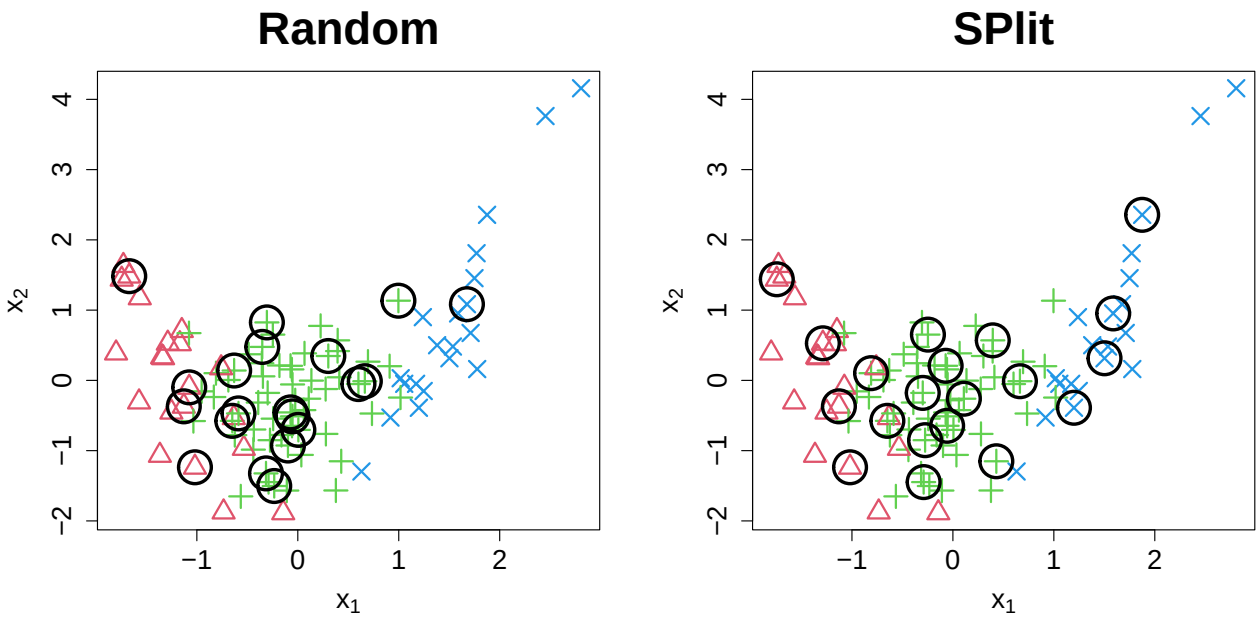
```
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
```

```
ind <- sample(1:N, Ntest)

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

plot(x1, x2, pch = y + 1, col = y + 1, cex.lab = 1.5, main = "Random",
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.main = 2.5, cex.axis = 1.5, cex = 2, lwd = 2)
points(D[ind, ], col = 1, pch = 1, lwd = 3, cex = 4)

Dtest <- D[ind.opt, 1:2]
plot(x1, x2, pch = y + 1, col = y + 1, cex.lab = 1.5, main = "SPlit",
     xlab = expression(x[1]), ylab = expression(x[2]),
     cex.main = 2.5, cex.axis = 1.5, cex = 2, lwd = 2)
points(Dtest, col = 1, pch = 1, lwd = 3, cex = 4)
```




```
par(mfrow = c(1, 1))
```

4 Section 10.3: Data Twinning

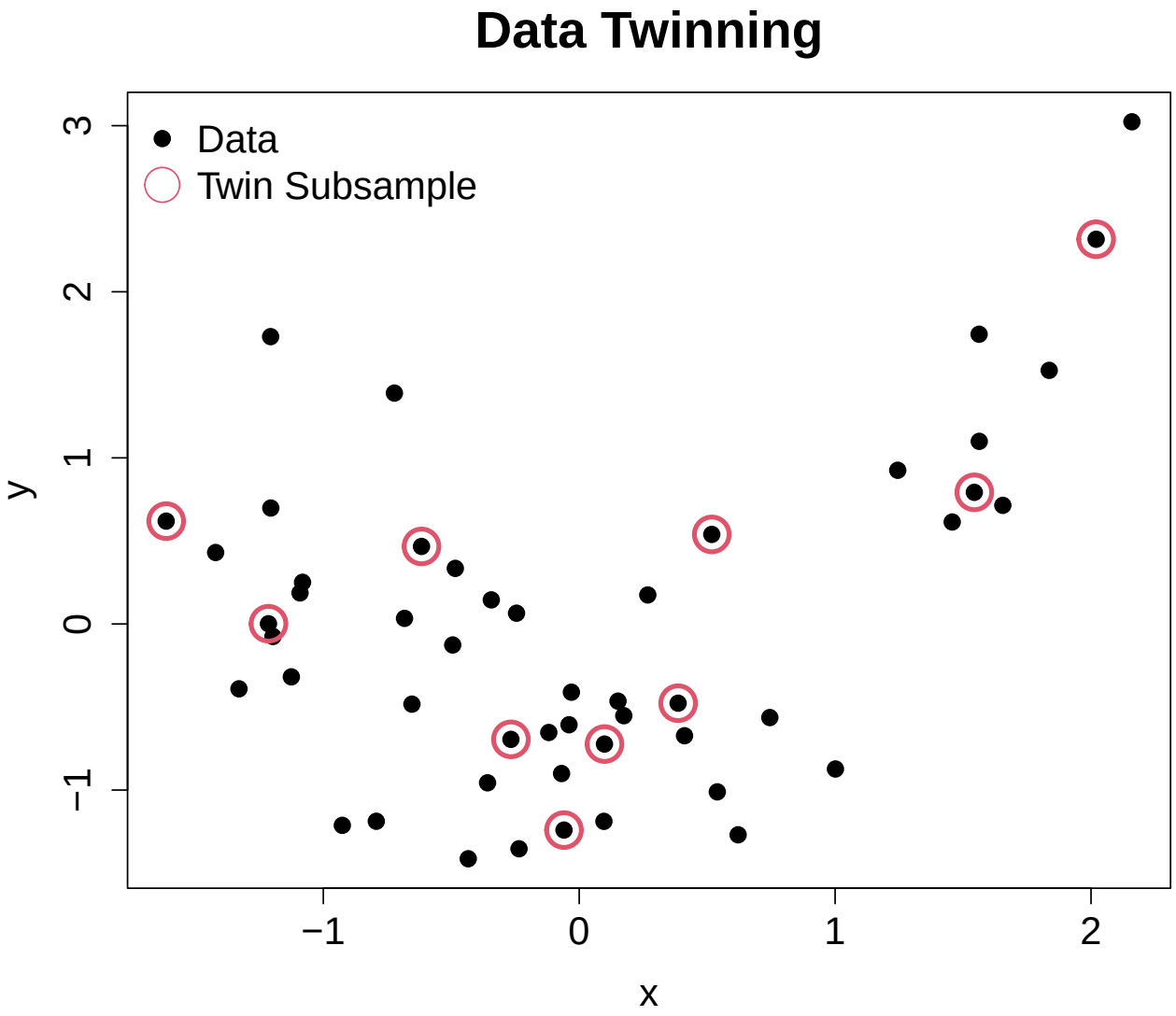
4.1 Basic Twinning Example

```
set.seed(6)
N <- 50
x <- scale(rnorm(N))
y <- scale(x^2 + rnorm(N))
D <- cbind(x, y)

ind.twin <- twin(D, r = 5)

plot(D, pch = 16, cex = 1.5, xlab = "x", ylab = "y",
     main = "Data Twinning", cex.main = 2, cex.lab = 1.5, cex.axis = 1.5)
```

```
points(D[ind.twin, ], col = 2, cex = 3, lwd = 3)
legend("topleft", legend = c("Data", "Twin Subsample"),
       pch = c(16, 1), col = c(1, 2), cex = 1.5, pt.cex = c(1.5, 3), bty = "n")
```



4.2 Comparison of Subsampling Methods

```
set.seed(4)
N <- 1000
X <- matrix(rnorm(2 * N), ncol = 2)
n <- 100

edsplit <- edtwin <- edlpm1 <- edlpm2 <- numeric(30)
sbsplit <- sbtwin <- sblpm1 <- sblpm2 <- numeric(30)

for (i in 1:30) {
  ind <- SPlit(X, splitRatio = n / N)
  edsplit[i] <- energy(X, X[ind, ])
  sbsplit[i] <- sb(rep(n / N, N), X, ind)
```

```
  ind <- twin(X, r = N / n)
  edtwin[i] <- energy(X, X[ind, ])
  sbtwin[i] <- sb(rep(n / N, N), X, ind)

  ind <- lpm1(rep(n / N, N), X)
  edlpm1[i] <- energy(X, X[ind, ])
  sblpm1[i] <- sb(rep(n / N, N), X, ind)

  ind <- lpm2(rep(n / N, N), X)
  edlpm2[i] <- energy(X, X[ind, ])
  sblpm2[i] <- sb(rep(n / N, N), X, ind)
}
```

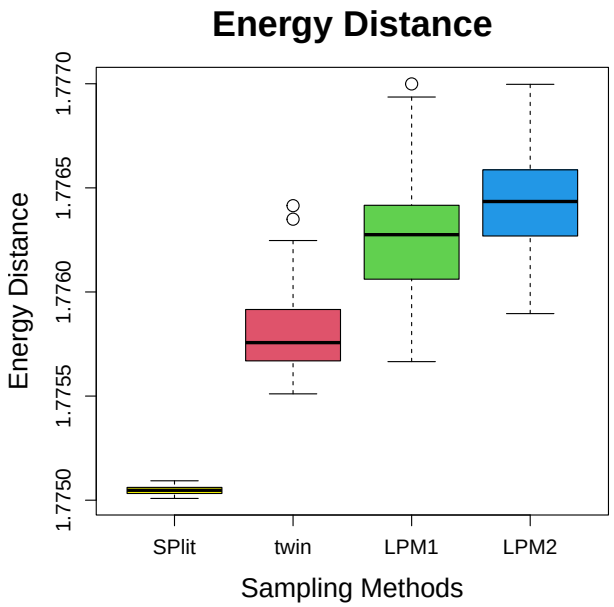
```
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++      ] 75%Optimizing <1 thread> [++++++++++++++      ] 76%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++        ] 62%Optimizing <1 thread> [++++++++++++        ] 63%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++      ] 72%Optimizing <1 thread> [++++++++++++++      ] 73%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++               ] 29%Optimizing <1 thread> [++++++              ] 30%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++          ] 58%Optimizing <1 thread> [++++++++++          ] 59%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++                ] 23%Optimizing <1 thread> [++++                ] 24%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++          ] 56%Optimizing <1 thread> [++++++++++          ] 57%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++                ] 23%Optimizing <1 thread> [++++                ] 24%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++               ] 28%Optimizing <1 thread> [+++++               ] 29%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++++++  ] 93%Optimizing <1 thread> [++++++++++++++++++  ] 94%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++++++           ] 45%Optimizing <1 thread> [+++++++++           ] 46%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++                 ] 18%Optimizing <1 thread> [+++                 ] 19%Optim
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimi
```

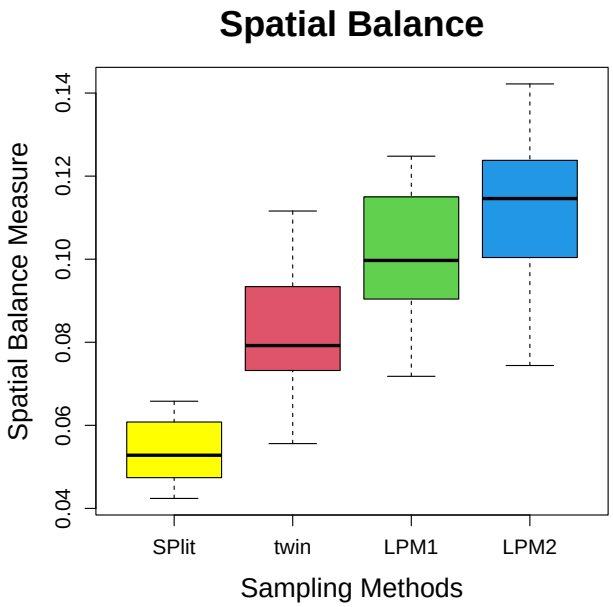
```
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++++             ] 38%Optimizing <1 thread> [+++++++             ] 39%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++        ] 63%Optimizing <1 thread> [++++++++++++        ] 64%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++                ] 22%Optimizing <1 thread> [++++                ] 23%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++      ] 78%Optimizing <1 thread> [++++++++++++++      ] 79%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++++++  ] 94%Optimizing <1 thread> [++++++++++++++++++++ ] 95%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++                ] 23%Optimizing <1 thread> [++++                ] 24%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++++             ] 39%Optimizing <1 thread> [++++++++            ] 40%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++++++      ] 79%Optimizing <1 thread> [++++++++++++++++    ] 80%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [+++++               ] 29%Optimizing <1 thread> [++++++              ] 30%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++++          ] 51%Optimizing <1 thread> [++++++++++          ] 52%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++++            ] 43%Optimizing <1 thread> [++++++++            ] 44%Optir
## Optimizing <1 thread> [                    ] 1%Optimizing <1 thread> [                    ] 2%Optimiz
## Numerical instablities encountered; resetting optimization.
## Optimizing <1 thread> [++++++              ] 34%Optimizing <1 thread> [+++++++             ] 35%Optir
```

```r
par(mfrow = c(1, 2))

ed <- data.frame(SPlit = edsplit, twin = edtwin, LPM1 = edlpm1, LPM2 = edlpm2)
boxplot(ed, main = "Energy Distance", xlab = "Sampling Methods",
        ylab = "Energy Distance", cex.main = 2, cex.axis = 1.2, cex = 1.5,
        cex.lab = 1.5, col = c("yellow", 2:4))

sbmeasure <- data.frame(SPlit = sbsplit, twin = sbtwin, LPM1 = sblpm1, LPM2 = sblpm2)
boxplot(sbmeasure, main = "Spatial Balance", xlab = "Sampling Methods",
        ylab = "Spatial Balance Measure", cex.main = 2, cex.axis = 1.2, cex = 1.5,
        cex.lab = 1.5, col = c("yellow", 2:4))
```






```
par(mfrow = c(1, 1))
```

5 Section 10.4: Supervised Compression

```
library(supercompress)
library(deldir)
library(FNN)
library(fields)

f <- function(x) {
  p <- length(x)
  x <- pi * x
  val <- -sum(sin(x) * (sin((1:p) * x^2 / pi))^(20))
  return(val)
}

p <- 2
N <- 10000 * p

set.seed(1)
x <- NULL
for (i in 1:p) x <- cbind(x, runif(N))
y <- apply(x, 1, f) + 0.01 * rnorm(N)
n <- 200
true <- apply(x, 1, f)

# K-means (unsupervised)
a0 <- kmeans(x, n, iter.max = 100)
D0 <- a0$centers
cluster <- a0$cluster
y0 <- sapply(split(y, cluster), mean)
pred.nn <- knn.reg(D0, test = x, y0, k = 1)$pred
rmse_kmeans <- sqrt(mean((pred.nn - true)^2))
```

```r
# Supercompress (supervised)
a <- supercompress(n, x, y)
Dopt <- a$D
ybar <- a$ybar
pred.nn <- knn.reg(Dopt, test = x, ybar, k = 1)$pred
rmse_super <- sqrt(mean((pred.nn - true)^2))

cat("RMSE (K-means):", round(rmse_kmeans, 4), "\n")
```

```
## RMSE (K-means): 0.1182
```

```r
cat("RMSE (Supercompress):", round(rmse_super, 4), "\n")
```

```
## RMSE (Supercompress): 0.0754
```

6 Summary

This chapter covered:

1. **Support points-based subsampling** - Using energy distance for representative samples
2. **Data splitting (SPlit)** - Optimal train/test splits
3. **Data twinning** - Creating twin datasets for validation
4. **Subsampling method comparison** - SPlit, Twin, LPM1, LPM2
5. **Supervised compression** - Adaptive clustering based on response